CGE-P LAB NOTES

The Plain English Guide

Read this before the labs, or alongside them when something looks like magic.

The labs tell you what to type. This tells you what it means, where every piece came from, and which choices are actually yours to make. Nothing in the curriculum should be a string you paste because a guide said so. If you find something here that is still mysterious, that is a bug in this document, not a gap in you.

For reading. Code lines wrap to fit the page; copy code from docs/00_00_plain_english_guide.md, not the PDF.

GRC Engineering Club

Companion to Labs 0.1 through 7.1

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/00_00_plain_english_guide.md` in your clone, not from the PDF.

Contents

1. What the whole thing is for

Here is the entire curriculum in one sentence:

> **Build a system where an auditor can verify your security controls without talking to you, and without trusting you.**

That is it. Everything else is detail.

Think about how compliance normally works. An auditor asks for evidence. Someone takes a screenshot of a console. The screenshot goes into a folder. Six months later nobody can tell whether the setting is still there, whether the screenshot was taken from production or a test account, or whether it was edited. The auditor's only real option is to trust the person who handed it over.

Every piece of this course attacks one weakness in that story:

Weakness in the screenshot approach	What the course does instead	Which chapter
"Was it configured correctly?"	Write it as code, so the configuration *is* the document	Ch 2
"Is it still configured correctly?"	Check it automatically on every change	Ch 3

Weakness in the screenshot approach	What the course does instead	Which chapter
"Who checked, and when?"	Run the check in a pipeline that records itself	Ch 4
"Could someone have edited the evidence?"	Sign it and store it where deletion is impossible	Ch 4
"Is anything else broken?"	Turn on the platform's own monitoring	Ch 5
"Which control does this even satisfy?"	Publish a machine-readable map from control to evidence	Ch 6

By the end, an auditor reads one JSON file, follows a link, runs one command, and sees `CHAIN INTACT`. You are not in the room. That is the deliverable.

Why it is built on Amazon and Google

The controls are the constant. The resources are not. `SC-28` means "protect data at rest" whether you are on AWS, GCP, Azure, or a server in a closet. But the *code* that satisfies it looks completely different on each.

Doing the same control twice, in two clouds, is the fastest way to internalize that the control ID is the durable thing and the resource type is disposable. It is also why your Rego policies in Chapter 3 need separate AWS and GCP variants that share one control ID.

2. The vocabulary

Plain definitions for the words that get used as though everyone already knows them.

Control. A requirement, written down, that reduces risk. "Data must be encrypted at rest" is a control. It is not a technology. Twelve different technologies could satisfy it.

Control family. NIST groups controls by two-letter prefix. You will meet these:

Prefix	Family	Rough meaning
`AC`	Access Control	Who can reach what
`AU`	Audit and Accountability	Logging, and doing something with the logs
`CM`	Configuration Management	Known-good settings, and knowing what you have
`CP`	Contingency Planning	Backup and recovery
`IA`	Identification and Authentication	Proving who you are
`RA`	Risk Assessment	Scanning, finding weaknesses
`SC`	System and Communications Protection	Encryption, boundaries

Prefix	Family	Rough meaning
`SI`	System and Information Integrity	Making sure things have not been tampered with
`SR`	Supply Chain Risk Management	Trusting the things you install

Control ID. `SC-28` is the 28th control in the System and Communications Protection family. `SC-28(1)` with a number in parentheses is a **control enhancement**, a stricter version of the base control. `SC-28` says protect data at rest. `SC-28(1)` says do it with cryptography specifically.

Framework. A curated list of controls someone decided you need. NIST 800-53 is a catalog of about a thousand controls. HIPAA, SOC 2, and CMMC are different lists for different purposes. They overlap heavily, because there are only so many ways to secure a system.

Catalog, in OSCAL terms. The full library of controls, published as a file. NIST publishes 800-53 as JSON. You link to it; you do not copy it.

Profile. A subset of a catalog. "These 12 controls out of the thousand are the ones that apply to us."

Baseline. A pre-made profile. NIST publishes Low, Moderate, and High baselines. This course builds its own small profile instead, because a Moderate baseline is around 300 controls and that is not a lab.

Preventive vs detective control. A preventive control stops the bad thing from happening. A detective control tells you it happened. A validation rule that rejects a bad value at plan time is preventive. A Security Hub finding three hours after deployment is detective. Preventive is stronger and usually cheaper; detective catches what prevention missed. You want both, and you should always be able to say which of yours is which.

Evidence. Something that demonstrates a control is working. A screenshot is weak evidence. A signed JSON file whose hash you can recompute is strong evidence. The difference is whether verifying it requires trusting anybody.

Attestation. A statement that something is true, in a form a machine can read. When Lab 2.3 outputs `encryption_algorithm = "aws:kms"`, that is an attestation.

Chain of custody. Four properties, together: it has not been altered (integrity), you know who made it (authenticity), you know when (timeliness), and nothing was quietly removed (completeness). Lab 4.4 implements all four, and most real-world pipelines implement two.

Infrastructure as Code (IaC). Describing your servers, buckets, and permissions in text files instead of clicking in a console. The text file becomes reviewable, versionable, and diffable, which is why it can serve as evidence.

Policy as Code. Writing your rules as a program that inspects a proposed change and approves or rejects it. Your compliance requirements become software that runs.

Drift. When reality stops matching the code. Someone changed a setting in the console; your Terraform no longer describes what exists.

Idempotent. Running it twice produces the same result as running it once. Terraform is idempotent: apply the same code repeatedly and nothing further changes.

Confused deputy. An attack where you trick a trusted service into doing something on the attacker's behalf. The service has permission; the attacker does not; the attacker uses the service as a puppet. This is why so many policies in these labs carry `aws:SourceArn` and `aws:SourceAccount` conditions.

Pinning. Naming an exact version of something you depend on, rather than a range or a moving label. A commit SHA for a GitHub Action, an exact version in `requirements.txt`, a provider hash in `.terraform.lock.hcl`. It buys you reproducibility and protection from an upstream changing under you, and it costs you the upstream's fixes until you deliberately move. See section 5's note on Chapter 4: pinning without a refresh process trades one risk for a quieter one.

Blast radius. How much breaks when something goes wrong. A sandbox account has a small blast radius, which is why Lab 0.1 insists on one.

3. The machinery around the labs

None of this is compliance. It is the scaffolding the labs stand on, and it is where most people lose their first evening, so it is worth understanding rather than pattern-matching.

3.1 The container, and why there is one

The labs use ten command-line tools. Installing them by hand means ten downloads, each with a different naming convention, and the instructions are written for one operating system on one processor architecture. Every Mac sold since 2020 uses a different architecture from the one those instructions assume, so every download line is subtly wrong on a Mac and completely wrong on Windows.

A **container** is a packaged filesystem with the tools already in it. You run it, and you are in a shell where `terraform` and `aws` already exist at the exact versions this curriculum was written against. Nothing is installed on your own machine, and nothing you do inside it can install anything on your own machine either.

Two ways to get one, from the same definition:

- **A Codespace** is that container running on GitHub's machines, with VS Code in a browser tab. Nothing is installed locally at all.
- **Locally**, Docker runs the same container on your machine and VS Code attaches to it.

The thing worth knowing, because it looks like magic and is not: the repository folder is *mounted* into the container rather than copied. Files you edit inside are your real files. Delete the container and your work is untouched.

Your cloud credentials work the other way round, and deliberately so. The container gets its own `~/.aws` on a separate volume rather than seeing yours. That is the safer arrangement: a real `~/.aws` collects every profile you have ever configured, work and production included, and sharing it with a container shares all of them with everything running in there. The container needs one sandbox account, so it gets exactly that.

The cost is that you log in twice, once on your machine and once inside, and that is the correct trade rather than an inconvenience to engineer away.

3.2 Why logging in from a container is different

`aws login` and `gcloud auth login` both work by opening a web browser. A container has no browser and no way to open one on your machine, so both hang or fail in ways that do not mention browsers at all.

Every such tool has a flag for this, because the problem is universal:

Command	Flag
`aws login`	`--remote`
`gcloud auth login`	`--no-launch-browser`
`gcloud auth application-default login`	`--no-launch-browser`

They all do the same thing: print a URL for you to open yourself, then wait while you paste back the code it gives you. The gcloud pair catches people because both commands need the flag, and the second is easy to forget.

3.3 How Terraform finds its inputs

A Terraform variable declared without a `default` has to come from somewhere. There are three somewheres, and knowing all three explains most confusing behavior:

How	Looks like	When it is used
On the command line	`-var project_name=cgep-lab`	One-off, explicit, easy to forget
A file it reads automatically	`terraform.tfvars`	Per workspace
An environment variable	`TF_VAR_project_name=cgep-lab`	Everywhere, once

The third is why this curriculum uses a file called `cgep.env`. `project_name` is an input to four different labs and `aws_region` to five. Typing them into each is how you end up with a bucket in the wrong region, or two labs that disagree about what your project is called, and neither mistake announces itself.

So you set them once, in a file you `source` at the start of a session, and Terraform picks them up on its own. There is no magic in it: Terraform reads any environment variable beginning `TF_VAR_` and strips the prefix to get the variable name.

If Terraform asks you for a value interactively, that is the tell. It means none of the three routes supplied it. Answering the prompt works once and teaches you nothing; the fix is to work out which route you meant to use.

`cgep.env` is deliberately not committed. It names your account, your project and your repository. It is configuration rather than a credential, but it is still yours.

3.4 Why the copy has to be yours

The capstone is graded on a public GitHub repository that you submit by URL. So your work has to live under your account from the first commit.

Cloning someone else's repository gives you their code pointed at *their* remote. Everything works right up until your first `git push`, which is refused, usually at the moment you have something worth keeping.

Use this template creates a fresh repository under your account with no relationship to the original. **Fork** does the same job but records that it came from somewhere else, which is fine for contributing back and slightly odd for work you are presenting as your own.

`git remote -v` tells you where a push would go. If your username is not in that URL, you are about to be refused.

Two things that are separate and are routinely confused: **your git identity** (`git config user.name` and `user.email`, which is what commits are labeled with) and **your authentication** (which is what proves you may push). Setting the first does not give you the second. Password authentication over HTTPS stopped working years ago, so `gh auth login` is the short path, and inside a container you want its device-code option for the reason in 3.2.

3.5 Repository variables are not secrets

GitHub gives a repository two places to keep values, and the difference matters more than it looks.

A **variable** is configuration. You can read it back, it shows up in logs, and that is a feature when a workflow fails and you need to see what it actually used. A role ARN and a bucket name belong here: they *identify* things, they do not grant access to them.

A **secret** is write-only and masked in logs. That is correct for a credential and unhelpful for anything else. Putting non-secrets in `secrets` is a common habit, and it quietly teaches that everything is equally sensitive, which is precisely how genuine credentials end up handled casually.

The stronger point is what the pipeline lab stores in neither: **no access key, anywhere**. The OIDC trust means GitHub proves which repository and which branch is asking, and AWS hands back credentials that last minutes. There is no long-lived secret to leak, rotate, or accidentally print. A pipeline that pastes an access key into `secrets` is doing the same job far less safely, and the difference is the entire lesson of Lab 4.3.

3.6 What a saved plan is, and why it goes stale

Most labs run `terraform plan -out=tfplan` and then `terraform apply tfplan`, rather than a bare `apply`. The reason is worth understanding, because it is the same reason it sometimes refuses to run.

A bare `terraform apply` computes a plan and immediately carries it out. Passing a saved plan splits that in two: the plan is a **file recording exactly which API calls Terraform intends to make**, and apply performs precisely those and nothing else. What you reviewed is what runs. In a pipeline that difference is the whole control, and it is why Lab 4.3 gates on a plan rather than on an apply.

The cost is that a saved plan is a promise about a particular state. If the state moves after the plan is written, the promise no longer holds, and Terraform says so:

```
Error: Saved plan is stale
The given plan file can no longer be applied because the state was changed
by another operation after the plan was created.
```

That is the safety feature working, not a fault. Something touched the state between your plan and your apply: another `terraform` command in that workspace, a colleague, a pipeline, or simply a `terraform output` or `refresh` that recorded a data source. Terraform cannot tell a harmless change from a dangerous one, so it refuses rather than guessing.

The fix is always the same: plan again, read it again, apply again. Nothing is broken and nothing is lost. If the new plan differs from the old one, that difference is exactly what you were being protected from, so read it rather than skim it.

Two habits that avoid it: do not leave a saved plan sitting for hours before applying it, and do not run Terraform against the same workspace from two places at once.

3.7 What is safe to publish

Your evidence *is* your portfolio, so most of it is meant to be public. Three things to be deliberate about:

Terraform state is the real risk. State records every attribute the provider stored, including ones marked sensitive, so a workspace that manages a database password has that password in its state. This is exactly the trap Lab 2.5 is built around. Look before you publish, and publish on purpose.

Your account ID will be in there, in every ARN, unavoidably. It is not a credential and publishing it is not a breach, but it does help someone enumerate your roles. Decide once whether you are comfortable with that; if not, keep the repository private and share it with reviewers directly.

Bucket names are globally unique and yours. Publishing them tells the world which buckets to probe. The controls you built in Lab 2.3 are exactly what makes that survivable, which is worth saying in your write-up rather than hiding.

4. The decoder ring

This section exists because of one line in the brief for this guide: nothing should be a mystery as to where it comes from. Every odd-looking string in the labs is below, with its source.

4.1 Amazon Resource Names (ARNs)

An ARN is AWS's universal address for a thing. The shape:

```
arn:partition:service:region:account-id:resource
 |     |        |       |        |         |
 |     |        |       |        |         +-- what it is
 |     |        |       |        +------------ whose it is (12 digits)
 |     |        |       +---------------------- where it lives
 |     |        +------------------------------ which AWS service
 |     +--------------------------------------- aws, aws-us-gov, or aws-cn
 +--------------------------------------------- always the literal "arn"
```

So why do S3 buckets look like this, with two empty fields?

```
arn:aws:s3:::my-bucket-name
```

Because **S3 bucket names are globally unique across every AWS account on Earth.** There is only one `my-bucket-name` anywhere. It needs no region and no account number to be unambiguous. That emptiness is not a typo, it is a statement about how S3 naming works.

Compare a KMS key, which is regional and account-scoped, so every field is populated:

```
arn:aws:kms:us-east-1:123456789012:key/1234abcd-12ab-34cd-56ef-1234567890ab
```

Why the labs use `arn:${local.partition}:...` instead of typing `arn:aws:`. Most of the world is in the `aws` partition. GovCloud is `aws-us-gov` and China is `aws-cn`, and an ARN with the wrong partition silently matches nothing. The `data "aws_partition" "current" {}` data source asks AWS which partition you are actually in. It costs nothing and makes the code correct everywhere.

4.2 `arn:aws:iam::123456789012:root` does not mean the root user

This is the single most misread string in the curriculum.

In a **KMS key policy** or a **bucket policy**, `arn:aws:iam::ACCOUNT_ID:root` means **"the AWS account itself,"** as a container. It is shorthand for "any principal in this account that also has IAM permissions allowing the action." It does **not** mean the root login.

That is why the labs put this in every key policy:

```
statement {
  sid       = "EnableAccountRootAdmin"
  effect    = "Allow"
  actions   = ["kms:*"]
  resources = ["*"]
  principals {
    type        = "AWS"
    identifiers = ["arn:aws:iam::123456789012:root"]
  }
}
```

Without it, the key is only usable by whoever the policy explicitly names, and **there is no way to add anybody later**, because changing the key policy requires permission that only the key policy can grant. You have created a key nobody can administer. AWS lets you do this. It is not recoverable.

Meanwhile, in Lab 0.1 when we say "harden the root user, enable MFA, never use it again," that is the actual root login, a different thing entirely, identified by an email address rather than an ARN.

And that second meaning has a trap. `aws login` reuses whatever console session you happen to be signed into, and it will offer you the **root** session without flagging that as a bad idea. Accept it and every `terraform apply` in this curriculum runs as root: unrestricted, unable to be constrained by any IAM policy, and impossible to scope down afterward.

It also quietly voids the exercise. Lab 4.3's OIDC role is scoped so CI can plan but not apply. Lab 2.5's vault denies `s3:DeleteBucket` to everyone except the account root. Lab 2.2 grants the account root key administration as a safety net. Operate as root by default and all three become decorative, and you spend four chapters generating evidence that controls work when nothing was ever subject to them.

Sign in as a normal IAM user first, then check:

```
aws sts get-caller-identity
```

If the `Arn` ends in `:root`, stop and fix it before applying anything.

4.3 Service principals

Strings like `logging.s3.amazonaws.com` are **service principals**: the identity a piece of AWS assumes when it acts on your behalf.

The ones in this curriculum:

Principal	Who it is	Appears in
`logging.s3.amazonaws.com`	S3's server access logging system, writing your log files	Lab 2.3
`cloudtrail.amazonaws.com`	CloudTrail, writing trail files	Lab 5.2
`cloud-storage-analytics@google.com`	Google's equivalent, a fixed group, not a service account you create	Lab 2.4

Where do you find these? You do not derive them or guess them. AWS documents the correct principal in the page for each feature, and using the wrong one produces a permission error rather than a security hole. The Google one is a literal, unchanging group address that Google publishes.

The point to internalize: when a bucket policy says "allow `logging.s3.amazonaws.com` to PutObject," you are not granting access to a person or a role. You are granting it to a robot inside AWS that will act only when a specific feature is switched on.

4.4 Condition keys

Conditions are how an IAM policy says "only when." There are two kinds.

Global condition keys start with `aws:` and work on any service:

Key	Plain meaning	Where used
`aws:SecureTransport`	Was this request made over HTTPS? `false` means plain HTTP	The TLS deny in every bucket policy
`aws:SourceArn`	Which specific resource caused a service to make this call	Confused-deputy protection
`aws:SourceAccount`	Which account owns the thing that caused the call	Confused-deputy protection
`aws:PrincipalArn`	Who is asking	The "only root may delete this bucket" rule in Lab 2.5

Service-specific keys start with the service name:

Key	Where it comes from
`s3:x-amz-server-side-encryption`	Literally the HTTP header `x-amz-server-side-encryption` on the upload request
`s3:x-amz-server-side-encryption-aws-kms-key-id`	The header naming which KMS key

That second group is worth pausing on: those keys exist because S3's API is HTTP, and the condition is inspecting a header on the actual request. It is not abstract. If you ran the upload with `curl`, you would be setting that header yourself.

The trap this creates. When a Deny statement uses `StringNotEquals` on a header, and the request does not send that header at all, IAM evaluates "not equals" as **true**, so the Deny fires. That is why Lab 2.3 warns that a plain `aws s3 cp` gets rejected: the file would have been encrypted anyway by the bucket default, but the request did not *say* so, and the policy demands that it say so.

4.5 The GitHub OIDC strings

Chapter 4 replaces stored AWS keys with a trust relationship. Three strings do the work.

`https://token.actions.githubusercontent.com` is **GitHub's OIDC issuer**. It is the URL where GitHub publishes the keys that prove a token really came from GitHub Actions. AWS fetches from it to verify signatures.

`sts.amazonaws.com` is the **audience**: who the token is intended for. A token minted for one audience cannot be replayed against another. AWS refuses tokens not addressed to it.

`repo:OWNER/REPO:ref:refs/heads/main` is the **subject claim**, a string GitHub builds and puts inside the token. Its format is GitHub's, not AWS's, and it varies by trigger:

Trigger	`sub` looks like
Push to a branch	`repo:OWNER/REPO:ref:refs/heads/main`
Pull request	`repo:OWNER/REPO:pull_request`
Deployment environment	`repo:OWNER/REPO:environment:production`
Tag	`repo:OWNER/REPO:ref:refs/tags/v1.0`

This is why the trust policy uses `StringLike` with `repo:OWNER/REPO:*`: it covers every trigger. And it is why the capstone can bind the *apply* role to `:environment:production` specifically, so only a job running in a protected environment gets those credentials. **The wildcard you must never write is `repo:*:*`, which trusts every repository on GitHub.**

The `thumbprint_list` value `6938fd4d98bab03faadb97b34396831e3780aea1` is a fingerprint of GitHub's certificate chain. AWS now maintains this trust internally for the GitHub provider, so the value is no longer load-bearing, but the argument is still required. It is one of the few genuinely vestigial strings in the curriculum, and worth knowing is vestigial so you do not worry about rotating it.

4.6 `resources = ["*"]` inside a key policy

This confuses everyone exactly once.

In an IAM policy attached to a user or role, `Resource: "*"` means "every resource in the account." Alarming.

In a **KMS key policy**, the policy is attached to one specific key, and `"*"` means **"this key."** It cannot mean anything else, because a key policy has no reach beyond its own key. Seeing `kms:*` on `"*"` in a key policy is normal and correct.

4.7 S3 setting names

Value	What it actually does
`BucketOwnerEnforced`	ACLs are off entirely. Only IAM and bucket policies decide access. **The modern default and what the labs use.**
`BucketOwnerPreferred`	ACLs still work, but new objects belong to the bucket owner. Tempting, because it lets you apply a log-delivery ACL.
`ObjectWriter`	The uploader owns what they upload. The old default, and a way to end up with objects you cannot read in your own bucket.
`AES256`	S3 manages the key. Free, no configuration, nothing for you to rotate or audit.
`aws:kms`	A KMS key manages it. Costs $1/month, and you control rotation, policy, and who can decrypt.
`GOVERNANCE`	Object Lock retention that a sufficiently privileged caller can bypass
`COMPLIANCE`	Object Lock retention that **nobody** can bypass, including the account root, until it expires

Value	What it actually does
`GLACIER_IR`	"Instant Retrieval" archive storage. Cheaper to store, costs more per read, no retrieval delay. Right for logs.

4.8 Terraform syntax that looks like magic

Where do attribute names come from? When you write `aws_s3_bucket.primary.arn`, the `.arn` is defined by the **provider schema**, not by Terraform. The AWS provider publishes what each resource exports. You can read it locally:

```
terraform providers schema -json | jq '.provider_schemas[].resource_schemas.aws_s3_bucket.block.attributes | keys'
```

That is the authoritative answer to "what can I put after the dot," and it beats guessing.

`aws_s3_bucket.primary.id` vs `.bucket` vs `.arn`. For this resource, `id` and `bucket` are both the bucket name and are interchangeable. `arn` is the full ARN. Different resources make different choices about what `id` means, which is why the labs prefer the explicitly named attribute where one exists.

`~> 5.0` is the "pessimistic constraint operator." It means "at least 5.0, and let the rightmost specified part increase, but nothing beyond." So `~> 5.0` allows any 5.x and refuses 6.0. `~> 5.100.0` allows 5.100.1 but not 5.101.0. It exists because major versions carry breaking changes and minor ones should not.

`local.` vs `var.` vs `data.`

Prefix	What it is
`var.`	An input somebody gives you
`local.`	A value you computed from other values
`data.`	A fact you looked up from the provider, not something you created

`data "aws_caller_identity" "current" {}` asks AWS "who am I?" and returns your account ID. It creates nothing. It exists so you never hardcode your account number.

`depends_on`. Terraform normally works out order by itself: if resource B mentions resource A, A goes first. `depends_on` is for the case where the ordering is real but invisible, because B never mentions A. In Lab 2.3, the logging configuration needs the log bucket's *policy* to exist first, but it only references the *bucket*. Terraform cannot see that, so you tell it. Most `depends_on` in the wild is cargo cult; the two in these labs are not.

`one([for ... ])`. Terraform models some nested blocks as a **set**, and sets have no order, so you cannot ask for element zero. The `for` turns the set into an ordered list, then `one()` pulls out the single element and **raises an error if there is more than one**. The alternative, `tolist(...)[0]`, silently picks an arbitrary one. Since this value goes into a compliance attestation, silently picking is the worse failure.

`coalesce`, `merge`, `alltrue`. `coalesce(a, b)` returns the first that is neither null nor empty. `merge(x, y)` combines maps with **y winning on conflicts**, which is exactly why Lab 2.4 writes `merge(var.labels, local.required_labels)` and a consumer cannot override a compliance label. `alltrue([...])` is true only if every element is.

`filter {}`**, empty.** In a lifecycle rule this means "apply to every object." The provider requires either a filter or a prefix, and an empty filter is how you say "no filtering."

4.9 Rego syntax

`package compliance.sc28` is a namespace. When Conftest runs with `--namespace compliance.sc28`, that string must match exactly. That is the whole relationship.

`deny contains msg if { ... }` builds a **set** of denial messages. Every combination of values that satisfies the body adds one message. An empty set means no violations. It reads strangely at first because you are not writing "if bad then fail," you are describing what a violation *looks like* and letting the engine find all of them.

`input` is whatever JSON you fed in. In this course that is `terraform show -json`, so `input.planned_values.root_module.resources` walks the structure of a Terraform plan.

`import rego.v1` opts into the modern syntax. In OPA 1.0 and later this is the default and the line is harmless; in OPA 0.x it is required. Writing it explicitly means the same policy runs on both.

`# METADATA` blocks are not comments to OPA. It parses them as structured annotations you can extract with `opa inspect --annotations`. That is how Lab 6.1 generates OSCAL from your policies instead of retyping every control ID.

4.10 Sigstore

Signing usually means managing a private key, which means protecting it forever. Sigstore's keyless flow avoids that:

1. Your CI job proves who it is with an OIDC token, the same mechanism as the AWS trust in Chapter 4.
2. **Fulcio**, a certificate authority, issues a certificate that is valid for about ten minutes, containing your identity.
3. You sign, and the key is discarded.
4. **Rekor**, a public append-only transparency log, records the signature and the time.

Nobody stores a long-lived key, and the timestamp lives in a public log outside your infrastructure. That last part is the important one: **an attacker with full administrative access to your AWS account still cannot forge or backdate a Rekor entry**, because Rekor is not in your AWS account.

The `.sig.bundle` file packs the signature, the certificate, and the log entry together so verification needs one file.

"Containing your identity" is worth reading twice. In the pipeline, that identity is the repository, workflow and ref, which is what you want: it names the process that produced the artifact rather than whoever happened to be at a keyboard. But if you run `cosign sign-blob` from your own machine, it is the email address of the account you log in with, and it goes into Rekor.

Rekor being append-only is the property that makes signatures trustworthy, and it is also the property that means **you cannot take that back**. An email address you publish there is public permanently. Cosign warns you before it proceeds, and the warning is literal rather than boilerplate.

So sign from CI, where the identity is a workflow. If you want to try it by hand, use an address you are content to have publicly associated with a throwaway lab artifact forever. This is the rare case where the immutability the whole curriculum is built around works against you.

4.11 OSCAL

Five document types. You will use three.

Type	Plain English
Catalog	The dictionary of all controls. NIST publishes it; you link to it.
Profile	Which controls you picked.
Component Definition	How your thing satisfies those controls, with links to evidence.
SSP	The whole system's story. Out of scope here.
Assessment Plan/Results	The auditor's side. Out of scope here.

Why v4 UUIDs specifically. OSCAL requires the version-4 (random) format, recognizable by the `4` starting the third group: `xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx`. Generate them with `python3 -c "import uuid; print(uuid.uuid4())"`. Hand-written ones fail validation with an unhelpful regex error.

`implementation-status` is where honesty lives:

- `implemented` means your code does it and you can point at the line.
- `partial` means some of it, and you can say which part is missing.
- `inherited` means the cloud provider does it and you rely on that.

`inherited` is not a cop-out. On GCP, encryption in transit is genuinely provided by the platform because the storage API refuses plain HTTP. Writing `implemented` there would be a small lie that collapses the moment somebody asks to see the code.

4.12 How your credentials actually reach Terraform

This one is worth its own entry because the error message points nowhere near the cause.

The AWS CLI and Terraform do **not** share a credential mechanism. The CLI understands several ways of holding a session; Terraform's provider is built on the Go SDK and understands a much shorter list. When they disagree, the CLI works perfectly and Terraform claims you have no credentials at all.

`aws login` writes exactly this and nothing else:

```
[default]
login_session = <token reference>
region        = us-east-1
```

There is no `~/.aws/credentials` file. `login_session` is a CLI construct, the provider has never heard of it, so the provider walks its chain, finds nothing, and reaches its last resort: asking the EC2 instance metadata service whether it is running on an EC2 box with a role attached. It is not, so that call times out:

```
Error: No valid credential sources found
Error: failed to refresh cached credentials, no EC2 IMDS role found,
       operation error ec2imds: GetMetadata, request canceled,
       context deadline exceeded
```

Nothing is wrong with EC2. That is simply where the search gave up, three steps after the actual problem. The same shape of error appears with IAM Identity Center, for the same reason.

The fix is to hand the provider a profile that asks the CLI for credentials each time it needs them. In `~/.aws/config`:

```
[profile cgep]
region = us-east-1
credential_process = aws configure export-credentials --profile default --format process
```

Then, in every new terminal:

```
export AWS_PROFILE=cgep
```

`credential_process` is a plain part of the AWS SDK contract: the SDK runs the command, reads JSON with an access key, a session token and an `Expiration` back from it, and runs it again once that expiration passes. Terraform never learns what `aws login` is, and never needs to.

There is an obvious-looking alternative worth explaining, because you will see it recommended: `eval "$(aws configure export-credentials --format env)"`, which copies today's credentials into the shell. It works, briefly. The credentials it copies are a snapshot with roughly a fifteen-minute life, and nothing refreshes them. Worse, environment variables sit *above* profiles in the credential chain, so once that snapshot expires the shell is stuck in a state that logging in again does not repair: `aws login` reports success, Terraform keeps saying `ExpiredToken`, and the two facts look impossible together. They are not. The provider is still reading the dead variables and never reaches the profile. The cure is `unset`, not another login.

The profile approach has no snapshot to go stale, which is the entire reason this course uses it.

Two smaller things worth knowing before they worry you. Run any command in the second or two right after `aws login` and you may see *Credentials were refreshed, but the refreshed credentials are still expired*; the CLI hands your prompt back before its own cache settles, so just run it again. And when the session does lapse, log in with `aws login --profile default`, not a bare `aws login`: with `AWS_PROFILE=cgep` set, a bare login targets `cgep` and the CLI refuses, because it will not overwrite a profile that resolves credentials through a process.

The general rule: **any credential source that is not a static key file needs this bridge.** Which is every source you should actually be using. A static key file avoids the problem by being the thing you were trying not to have.

5. The numbers

Every constant in the curriculum, and where it came from.

`byte_length = 4` on the random suffix. Four bytes render as 8 hexadecimal characters, giving about 4.3 billion possibilities. Enough that no two learners collide; short enough to fit the name budget below.

21 characters maximum for `project_name`. This is derived, not chosen. S3 allows 63 characters. The longest name the labs build is `project` + `-` + `staging` + `-data-` + 8 hex = 21 + 1 + 7 + 6 + 8 = **43**, comfortably inside 63 with room for a longer suffix later. **Change the naming scheme and this number has to move.**

63 characters is S3's hard limit, and it comes from DNS: bucket names originally had to work as hostnames, and a DNS label maxes out at 63 characters.

`7776000s` for KMS rotation on GCP. That is 90 days in seconds. Google's API takes a duration string, so you do the arithmetic: 90 × 24 × 60 × 60.

7 to 30 days for the AWS KMS deletion window. AWS's allowed range. It exists because deleting a key destroys every piece of data it protects, permanently, and AWS wants a window in which you can change your mind. **The key bills the full $1/month throughout that window.**

90 days for access log retention, **400 days** for CloudTrail. Ninety is a common operational default. Four hundred is deliberate: a full year plus a margin, so that when an auditor asks in month thirteen about something from month one, the logs are still there.

$1 per month per KMS key. AWS's published price, charged whether or not you use the key. It is the largest recurring cost in the curriculum, and it is what buys you SC-12 and SC-13. An AWS-managed key is free and satisfies neither, because you can neither set its rotation policy nor read its key policy.

$0.10 per 100,000 events for CloudTrail data events. Management events are free; data events (who read which object) are not. This is why Lab 5.2 scopes them to the evidence vault only.

`0x2B` and `0x5A`. These are the ASCII codes for `+` and `Z`, and they explain a piece of Lab 4.4 that is subtler than it first looks. AWS returns retention dates ending `+00:00`; `date -u` produces dates ending `Z`. Comparing those as strings *looks* unsafe.

Test it and it holds up. Both strings share a fixed-width, zero-padded ISO prefix, so a lexicographic comparison of `YYYY-MM-DDTHH:MM:SS` is the same as a chronological one. The formats only diverge at the character after the seconds, and that character is only consulted when both sides are the *same second*, where "not in the future" is the right answer either way. Two hundred thousand generated cases produce zero disagreements.

What breaks it is a **non-UTC offset**. `2026-08-18T16:00:00-05:00` is an hour ahead of `2026-08-18T20:00:00Z`, and compares as expired. So the string test is correct by coincidence, not by construction: it depends on AWS returning UTC.

The lesson is not "string comparison is a bug." It is that code can be right for a reason you did not choose, and the fix is to make the reason explicit. Comparing epoch seconds is correct whatever format arrives.

Three of four public access block flags is not enough. Not a number so much as a shape. The four flags are two questions crossed with two states:

	Blocks *new*	**Neutralizes** *existing*
ACLs	`block_public_acls`	`ignore_public_acls`
Policies	`block_public_policy`	`restrict_public_buckets`

Leave one off and you have either an existing public grant still live or an open door for a new one.

6. The labs in plain English

Lab 0.1: Prerequisites

What it does. Sets up a throwaway AWS account and a GCP project, gets you credentials, installs about ten tools, and puts a spending cap in place.

Why it exists. Because the labs now create KMS keys that bill real money, and because the single most common way to lose an afternoon is an authentication error that looks like something else.

The credential decision. Three ways to let your laptop talk to AWS, and they are not equal. `aws login` converts the console session you already have into short-lived credentials and writes no lasting secret; it is the best default. IAM Identity Center gives the same property with more setup and is right for org-managed accounts. An IAM user access key is a permanent secret sitting in a file, and it is the credential type Chapters 4 and 5 exist to argue against. Whichever you pick, see section 3.12 for why Terraform will not see it until you export it.

The one thing not to skip: the budget alarm. It is the only step that has ever saved anyone money.

Lab 2.2: Remote state backend

What it does. Creates one S3 bucket whose job is to hold Terraform's memory.

What Terraform state actually is. When Terraform creates a bucket, it writes down what it made, so next time it can tell the difference between "create this" and "this already exists." That memory is a JSON file called state. Without it, Terraform has amnesia and tries to create everything again.

Why it has to be remote. Your laptop has the state file. Your CI runner does not. So the CI runner thinks nothing exists and proposes to build everything from scratch. Putting state in S3 lets both read the same memory.

Why it is treated as a secret. State contains **every attribute of every resource, in plaintext**, including database passwords and API keys. If you mark something `sensitive` in Terraform, that only hides it from the terminal output. It is still sitting in the state file in the clear. Read access to state is read access to your secrets.

The chicken-and-egg. You need a bucket to store state, and creating the bucket produces state. The answer: a small separate workspace that keeps its state locally and gets committed to git, because that particular state describes only a bucket and a key, with nothing secret in it. The lab shows you how to verify that claim rather than trust it.

Lab 2.3: Your first compliant resource

What it does. Builds two S3 buckets: one for data, one that receives the access logs of the first. Both are locked down eleven different ways.

Why two buckets. Logs should not live in the thing they are logging. If someone compromises the data bucket and can also rewrite its logs, the logs are worthless.

What each piece is for, in one line each:

Piece	Plain English
KMS key	An encryption key you own and can rotate, rather than one Amazon manages invisibly
Server-side encryption config	"Encrypt everything in this bucket with that key"
Versioning	Keep old copies, so a delete or overwrite is recoverable
Public access block	Four separate switches, all off, meaning nothing here goes public
Ownership controls	Turn off the old ACL permission system entirely
TLS deny policy	Refuse any request that arrives over plain HTTP
Encryption deny policy	Refuse any upload that does not explicitly ask for your key
Lifecycle rules	Delete logs after 90 days, so storage does not grow forever
Access logging	Write a record of every request into the other bucket
Tags	Four labels on everything, so you can list your whole environment with one API call

The tag that matters most. `ComplianceScope = "cge-p-lab"` lets you answer "what is in scope for this audit?" with a command instead of a spreadsheet. That is a genuinely different way to run compliance and it is the quiet point of the whole tagging exercise.

The one that will surprise you. After applying, `aws s3 cp file.txt s3://bucket/` **fails**. That is the encryption deny policy working. You have to say which key you want:

```
aws s3 cp file.txt s3://bucket/ --sse aws:kms --sse-kms-key-id "$KMS_ARN"
```

Whether that friction is worth the enforcement is a real decision, and defending your answer is exactly what the capstone asks for.

Lab 2.4: Modules

What it does. Repackages the same idea on Google Cloud as a reusable component, then uses it twice with different settings.

What a module is. A folder of infrastructure code with a defined set of inputs and outputs. Like a function. You hardcode the security settings inside where nobody can reach them, and expose only the business settings.

The design idea worth stealing. Required labels are merged **on top of** the consumer's labels, so a consumer can add labels but cannot override the compliance ones. That asymmetry, rather than documentation or code review, is what makes the floor a floor.

The structural lesson. This module deliberately contains no `provider` block, and Lab 2.3 deliberately does. A module with its own provider cannot be reused across two regions or two accounts. Lab 2.3's "primitive" is really a root module wearing the wrong word, and noticing that is the point.

Lab 2.5: The evidence vault

What it does. Creates a bucket where deletion is impossible, and a script that packages evidence and puts it there.

Object Lock in plain English. Normally a bucket owner can delete anything. Object Lock makes S3 itself refuse. In `GOVERNANCE` mode a sufficiently privileged caller can override it. In `COMPLIANCE` mode nobody can, including the account root, until the clock runs out.

The trap. `COMPLIANCE` mode with a long retention creates a bucket you will pay for and cannot empty for the entire duration. That is correct behavior and an expensive mistake in a lab. Use `GOVERNANCE` with one day while learning.

The trap. It is natural to capture raw Terraform state into this vault, since it describes what is actually deployed. Combine "state contains secrets in plaintext" with "this bucket cannot be deleted from" and you have made a secret permanently undeletable, in the bucket you hand to auditors. The lab captures the plan instead, and refuses to capture state into a COMPLIANCE vault without an explicit override.

A subtlety worth internalizing. Deleting an object without naming a version *appears* to work. It creates a "delete marker" that hides the object. The object is still there. An auditor who does not know this thinks evidence was destroyed; an engineer who does knows where to look.

Lab 3.3 and 3.4: Policy as code

What they do. Write programs that read a proposed infrastructure change and refuse it if it violates a control.

Why this is different from a scanner. A scanner tells you what is wrong after it exists. This runs on the *plan*, before anything is created. Nothing bad ever gets built.

Why Rego feels weird. Rego does not ask "is this OK?" It asks "show me every way this is broken," and returns a set. You describe the shape of a violation, and the engine finds all instances. It is closer to a database query than to an `if` statement.

Mutation testing, the most valuable idea in these two labs. A policy with a typo in the resource type matches nothing, denies nothing, and passes every test you wrote. It is a control that *cannot fail*, which means it is not a control. So you break each policy on purpose and require the tests to notice. If a test suite still passes with the logic inverted, that policy is decorative.

The same idea in one sentence: **a green pipeline looks identical whether your controls passed or never ran.**

The related trap. Running GCP policies against an AWS plan reports zero failures, because the rules match nothing. Zero failures and zero coverage look the same in the output. That is why Lab 3.4's gate script fails loudly when it produces no results at all.

Lab 4.3 and 4.4: The pipeline

What they do. Move the checks into GitHub Actions so they run on every proposed change, then sign the results and file them in the vault.

What OIDC replaces. The old way: create an access key, paste it into GitHub Secrets, hope it never leaks, remember to rotate it. The new way: GitHub proves the job's identity to AWS, AWS hands back credentials valid for an hour. **Nothing long-lived exists to leak.**

Why actions are pinned to a commit hash instead of `@v4`. A tag is a movable pointer. Whoever controls the repository, or whoever compromises their account, can move `v4` to point at different code. A commit hash cannot be moved. This is a real attack that has really happened.

And the part nobody mentions: a pin is a maintenance obligation. A tag floats, so it silently collects fixes and silently collects compromises. A hash is frozen, so it collects neither, including the upstream's runtime migrations. Eventually you are running something the platform is deprecating underneath you.

This curriculum's own CI showed it. Every job passed, and every job carried `Node.js 20 is deprecated ... being forced to run on Node.js 24`. Nothing was broken; the pins were simply old enough that GitHub was migrating them off their target runtime, and the only signal was a warning inside a green run.

So pin, and pair it with something that raises the version: Dependabot, Renovate, a quarterly review, or a CI check that fails when a pin falls too far behind. **A pin without a refresh process does not remove the supply chain risk, it swaps it for an obsolescence risk that is quieter.** That is usually still the right trade. Make it knowingly, and write down who checks.

Why the pass/fail decision moves to the last step. You want evidence collected even when the gate fails. **A failed run is precisely the run whose evidence you will want later.**

Branch protection is what makes it a control. Without it, a red check is a suggestion anybody can merge past. With `enforce_admins: true`, it is enforcement. That toggle is the difference between CM-3 and a habit.

What signing adds that Object Lock does not. Object Lock proves the file has not changed. It does not prove who made it. Someone with admin in your account could create a *different* bucket, put a fabricated bundle in it, and point a careless auditor there. The signature ties the bundle to a specific repository and workflow, and the record of it lives in a public log outside your account.

Lab 5.2 and 5.4: The platform's own tools

What they do. Turn on the monitoring each cloud already provides, and then actually read the output.

The three services, plainly. CloudTrail records who did what. Config records what things looked like over time. Security Hub collects findings and normalizes them.

Where AU-6 finally gets earned. Collecting logs is `AU-3`. Keeping them the right length is `AU-11`. **Reviewing** them is `AU-6`, and that requires something that reads them, on a schedule, with a human whose name you can say. Lab 5.2 sets up Athena so you can query the trail. If you run those queries once by hand, the honest claim is `partial`, not `implemented`.

The GCP difference. Google's Org Policy rejects bad configuration at the API call. Not a finding afterward: the action does not happen. That is the strongest kind of control, and also the one that can break your engineering team on the day you enable it, which is why the lab teaches audit mode first and enforcement second.

Lab 6.1: OSCAL

What it does. Writes down, in a format a machine can read, which controls your thing satisfies and where the proof is.

Why bother. Because it turns an audit from a meeting into a traversal. The assessor opens your file, sees `SC-28`, follows the link, verifies the signature, and moves on.

The generation idea. Your Rego policies already declare which control they enforce. Rather than retyping those IDs into the OSCAL by hand, the lab extracts them programmatically. **Anywhere a mapping is typed twice is a place it will drift.**

Why the verification script matters. `trestle validate` checks that your document is well-formed. It does not check that a single link works. A component definition whose evidence URLs all 404 validates perfectly and proves nothing. That script found a real error while this curriculum was being written: the NIST catalog tag pinned in the first draft did not exist.

7. Every decision you have to make

The labs pick sensible defaults. These are the ones you should think about rather than accept.

Encryption: AWS-managed key or your own?

	`AES256` (S3-managed)	`aws:kms` (your key)
Cost	Free	$1/month per key
Rotation	Amazon's business	Yours, on your schedule
Who can decrypt	Anyone with S3 permission	Anyone with S3 **and** KMS permission
Audit trail	None for key use	Every use logged in CloudTrail
Cross-account sharing	Simple	Requires key policy work

Choose your own key when you are handling regulated data, need to prove rotation, need a second lock separate from S3 permissions, or expect an assessor to ask about key custody.

Choose AES256 when it is a genuine sandbox, cost matters more than control, or the data is not sensitive. Be aware that Trivy and most benchmarks flag AES256 at HIGH, so you will be explaining the choice.

Object Lock: GOVERNANCE or COMPLIANCE?

GOVERNANCE for anything you might need to clean up, and for all lab work. **COMPLIANCE** when a regulator requires that evidence be genuinely undeletable, and you have accepted that "genuinely" includes you.

Ask yourself one question: *if I put the wrong file in here, am I comfortable that it stays for the full retention period?* If the answer is no, you want GOVERNANCE.

Where do logs go: server access logs or CloudTrail data events?

	S3 server access logs	CloudTrail data events
Cost	Free (storage only)	$0.10 per 100k events
Delivery	Best effort, can drop records	Guaranteed
Timing	Hours	Minutes
Format	Space-separated text	JSON
Integrity proof	None	Digest files

Use CloudTrail data events for the buckets that matter, especially the evidence vault, where "who read this" is itself an audit question. **Use server access logs for everything else**, because free and imperfect beats expensive and perfect at scale.

Should evidence bundles include Terraform state?

No, by default. State holds secrets in plaintext and the vault cannot be emptied. The plan file tells an assessor what was configured, which is what they actually read.

Yes, if you have deliberately confirmed the workspace holds no secrets, and you need to demonstrate current deployed reality rather than intent. Say so in your write-up either way; a reviewer will ask.

Which framework for the capstone?

Pick	If
HIPAA Security Rule	The scenario's PHI angle interests you. Smallest control set, most prescriptive language, easiest to be thorough about.
SOC 2	You want the one you are most likely to meet commercially. Trust Services Criteria are broader and vaguer, which means more judgment and more to defend.
CMMC Level 2	You are aiming at federal work. Most prescriptive, maps closest to NIST 800-171, largest volume.

There is no wrong answer, and the grading is on how well you defend the choice. Pick the one you can argue about for a page.

Apply automatically on merge, or require manual approval?

Automatic is the stronger demonstration of engineering maturity and the harder thing to build safely. **Manual approval** is what most regulated organizations actually do.

Either is acceptable. What is not acceptable is automatic apply using the same credentials as the plan step. Split the roles.

One AWS account or two?

Two (workload plus a separate evidence account) is the real answer, because same-account evidence can be destroyed by whoever compromised the account. **One** is fine for a 30-day capstone. Name the gap in your write-up and you have handled it correctly.

8. How to find the answer yourself

The goal is that you stop needing this document. When something is unfamiliar:

A Terraform attribute you do not recognize:

```
terraform providers schema -json | jq '.provider_schemas[].resource_schemas.RESOURCE_NAME'
```

What a plan is actually doing:

```
terraform show -json tfplan | jq '.planned_values.root_module.resources[] | {address, type}'
```

Why an IAM decision went the way it did. The IAM Policy Simulator in the console, or `aws iam simulate-principal-policy`. IAM's rule is simple and worth memorizing: **an explicit Deny always wins, then an explicit Allow, and otherwise the answer is no.**

What a control actually says. The NIST catalog is a JSON file you can grep:

```
curl -s https://raw.githubusercontent.com/usnistgov/oscal-content/v1.2.1/nist.gov/SP800-53/rev5/json/
NIST_SP-800-53_rev5_catalog.json \
  | jq '.. | objects | select(.id? == "sc-28") | {id, title}'
```

What a Rego rule is doing:

```
opa eval -d policies -i plan.json 'data.compliance.sc28_aws' --format=pretty
```

Drop the `.deny` to see every rule in the package, including the helpers.

Whether a tool version is current:

```
curl -s https://api.github.com/repos/OWNER/REPO/releases/latest | jq -r .tag_name
```

What you actually have deployed, using the tag from Lab 2.3:

```
aws resourcegroupstaggingapi get-resources \
  --tag-filters Key=ComplianceScope,Values=cge-p-lab --profile cgep-lab
```

The three habits

If the details fade, keep these.

1. Watch every control deny something. Configuration proves intent. A refusal proves enforcement. Every lab ends by making a control say no, on purpose, and that transcript is better evidence than any settings dump.

2. Break your checks to prove they work. An unfired control and an absent control look identical in a green pipeline. Mutation-test your policies. Delete the policy directory and confirm the gate goes red. Tamper with a bundle and confirm verification fails.

3. Claim only what you can show. `partial` that you can defend beats `implemented` that falls apart in one question. The most common overclaim in this field is AU-6, audit review, asserted on the strength of having collected the logs. Collecting is AU-3 and AU-11. Reviewing is a person, a cadence and a record.

Revision history

These notes

- New document. The labs say what to type; this explains what it means, traces every constant and magic string to its source, and lays out which choices are genuinely yours.
- Section 3 covers the scaffolding rather than the compliance: the container and why browser logins fail inside it, the three places Terraform looks for an input, why your copy of the repository has to be yours, why repository variables are not secrets, and what is safe to publish. Most first evenings are lost in there rather than in the labs.

The official labs

Did not exist.